VULNERABILITY TESTING / LAB REPORT

OWASP BROKEN ACCESS CONTROL

Web Application Penetration Testing – Internship Report

Organization: IqraSec Academy
Platform: TryHackMe

COVER PAGE

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT)
Topic	OWASP Broken Access Control
Platform	TryHackMe
Number of Labs	1
Completion Date	[Enter actual completion date]

This report documents the objective, testing methodology, evidence, impact, remediation, and learning outcomes of the Broken Access Control lab.

Lab 1: OWASP Broken Access Control

1. Lab Name

OWASP Broken Access Control

Platform: TryHackMe
Room URL: https://tryhackme.com/room/owaspbrokenaccesscontrol

2. Objective

The objective of this lab is to understand and identify Broken Access Control vulnerabilities in web applications. The exercise focuses on evaluating whether users can access resources or perform actions beyond their assigned permissions.

The main objectives are:

- Understand the concept of Broken Access Control.
- Identify horizontal and vertical privilege escalation.
- Test whether users can access other users' resources.
- Examine URL parameters, request methods, and application endpoints.
- Understand the risks of missing server-side authorization checks.
- Learn appropriate prevention and remediation techniques.

3. Vulnerability / Concept

What is Broken Access Control?

Broken Access Control is a web application security vulnerability that occurs when an application fails to enforce the permissions associated with a user or role. As a result, an authenticated or unauthenticated user may access restricted resources or perform unauthorized actions.

Access control determines **who can access which resource and which actions they are permitted to perform**.

Types of Access Control Issues

1. Horizontal Privilege Escalation

A user accesses another user's data or resources while operating at the same privilege level.

Example: User A changes an account identifier in a request and obtains User B's private information.

2. Vertical Privilege Escalation

A user with lower privileges performs an action reserved for a higher-privileged role.

Example: A standard user accesses an administrative function without the required authorization.

3. Insecure Direct Object Reference (IDOR)

An application uses a user-controlled identifier to retrieve a resource but fails to verify whether the requesting user has permission to access it.

Example: Changing an account or document identifier exposes another user's information.

4. Missing Server-Side Authorization

The application relies on hidden buttons, frontend restrictions, or client-side checks instead of enforcing permissions on the server for every relevant request.

Common Indicators

- Sensitive information is accessible by changing an object identifier.
- Restricted pages or API endpoints respond to unauthorized users.

- Standard users can perform administrative actions.
- Direct requests bypass restrictions imposed by the user interface.
- The server does not verify resource ownership or user roles.

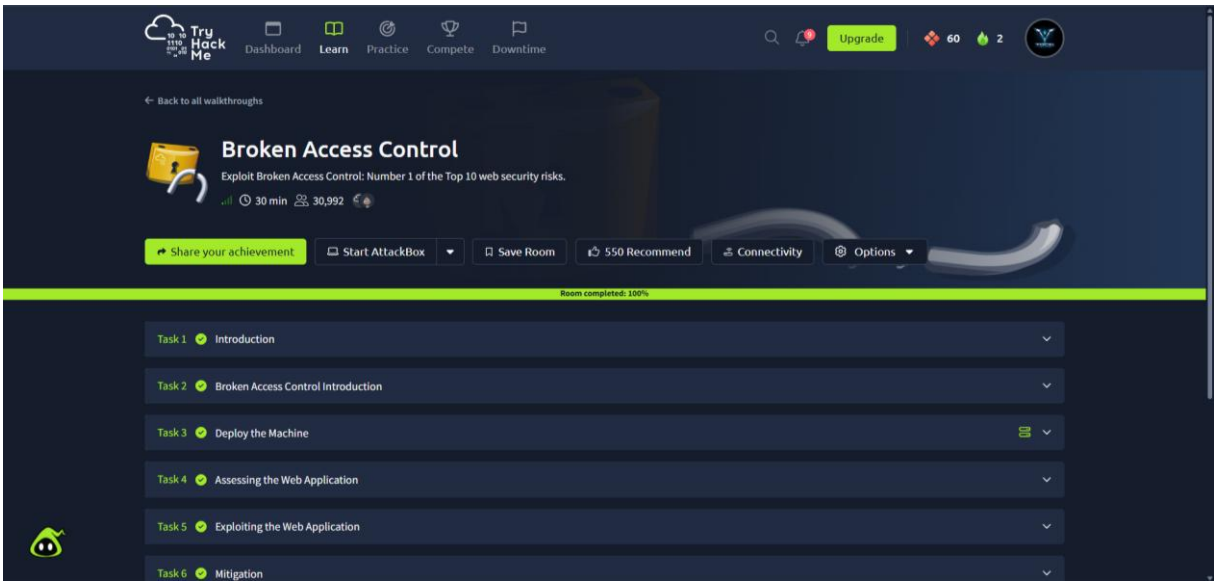
4. Testing Approach

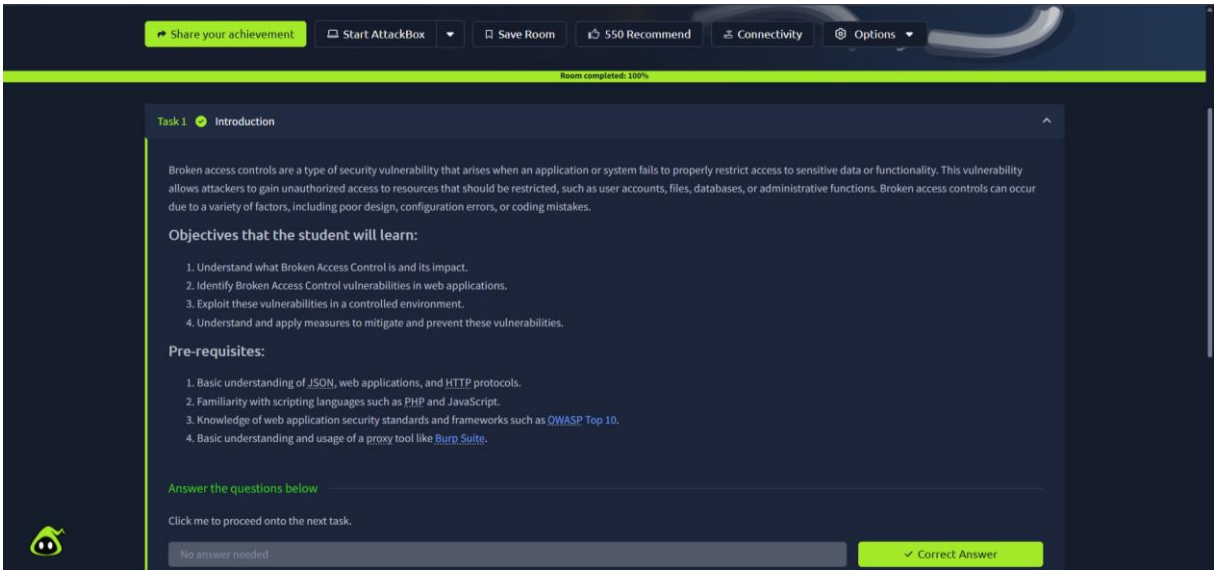
The following methodology provides a structured workflow for evaluating access control in an authorized lab environment.

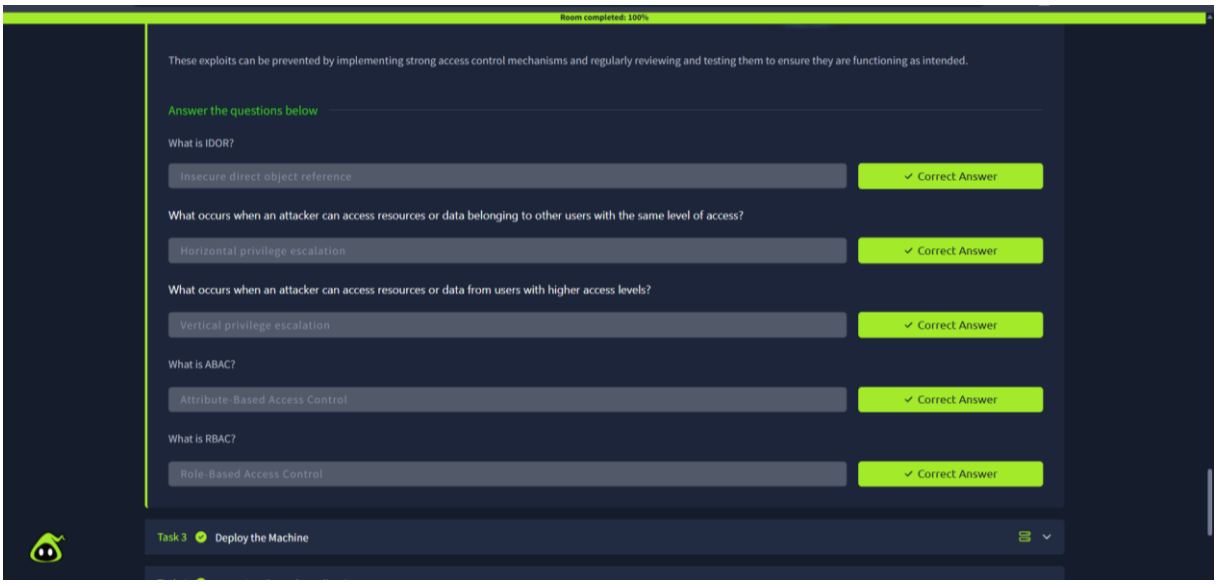
1. Open the TryHackMe room using the official room URL.
2. Read the room introduction and understand the lab objectives.
3. Start the provided lab environment, if required.
4. Open the target web application in the provided browser.
5. Review the application's pages, features, and available user actions.
6. Identify login functionality and the roles or permissions used by the application.
7. Authenticate with the test account supplied by the lab, if required.
8. Observe the URLs, query parameters, form fields, and API requests associated with resources.
9. Identify user-controlled object identifiers, such as account IDs or record IDs.
10. Establish a baseline by accessing a resource permitted to the current test user.
11. Within the lab, change an object identifier to test whether the server validates resource ownership.
12. Observe whether the response contains only authorized information or exposes another user's data.
13. Check whether a restricted endpoint can be accessed directly rather than through the application's navigation.
14. Evaluate whether actions available to higher-privileged roles are properly restricted.
15. Compare the behavior of requests made with different authorized test-user roles, where the lab provides them.
16. Observe HTTP response codes, response content, redirects, and error messages.
17. Record any confirmed unauthorized access, including the affected endpoint and relevant request details.
18. Capture screenshots showing the relevant test and the observed result.
19. Compare the findings against expected access control behavior.
20. Document the confirmed results and identify suitable remediation measures.

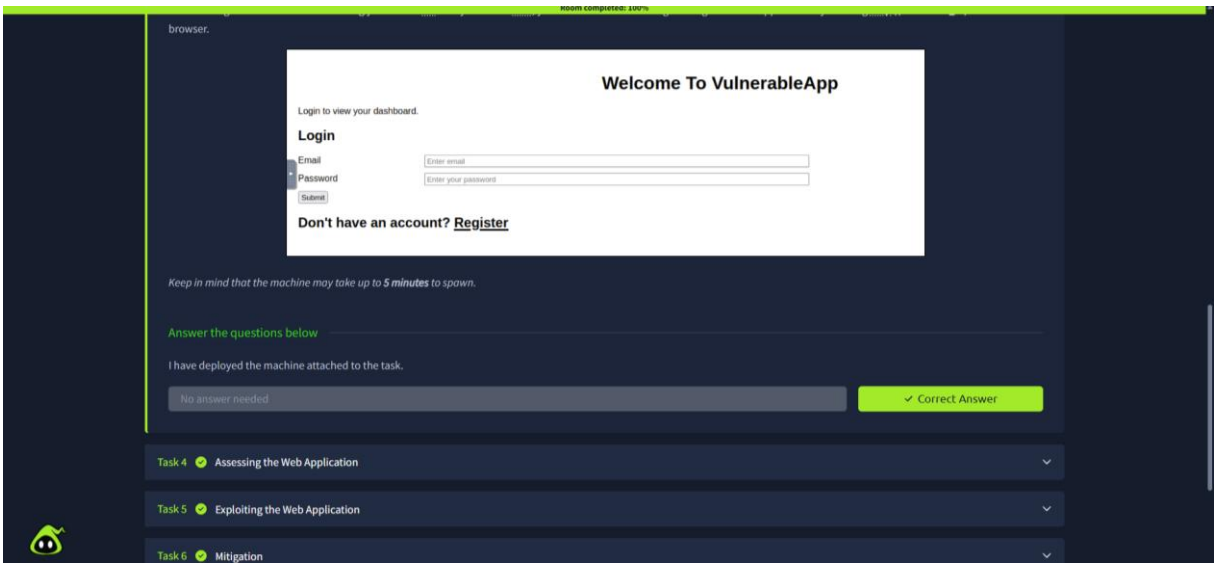
Testing note: Perform these checks only against the assigned TryHackMe target or another system for which explicit authorization has been granted. Record only the tests actually performed and the results actually observed.

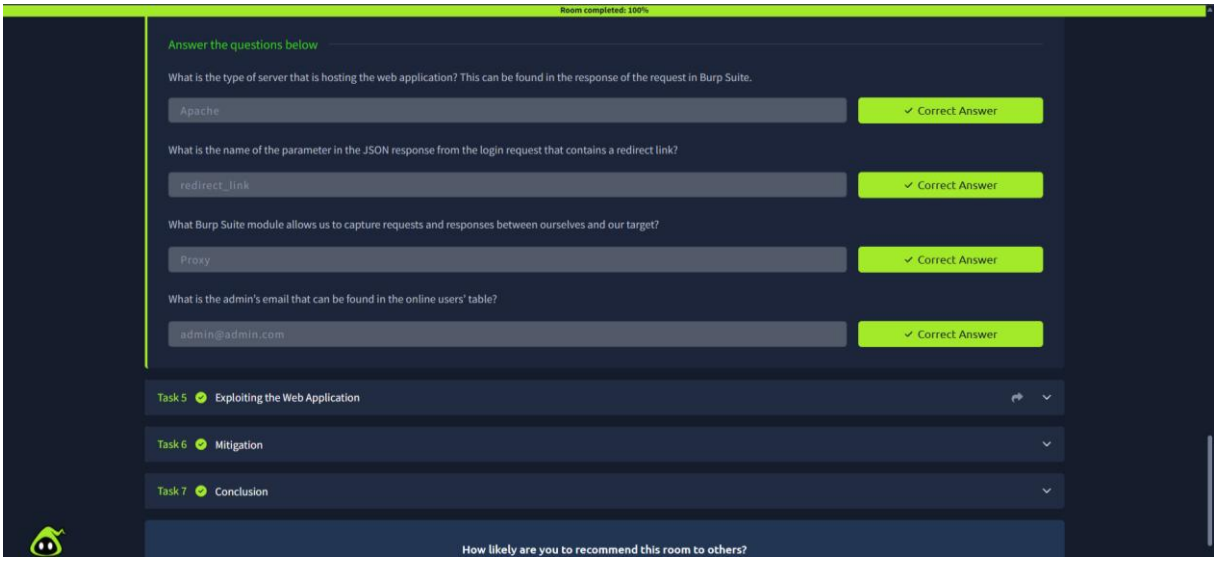
5. Evidence / Screenshots

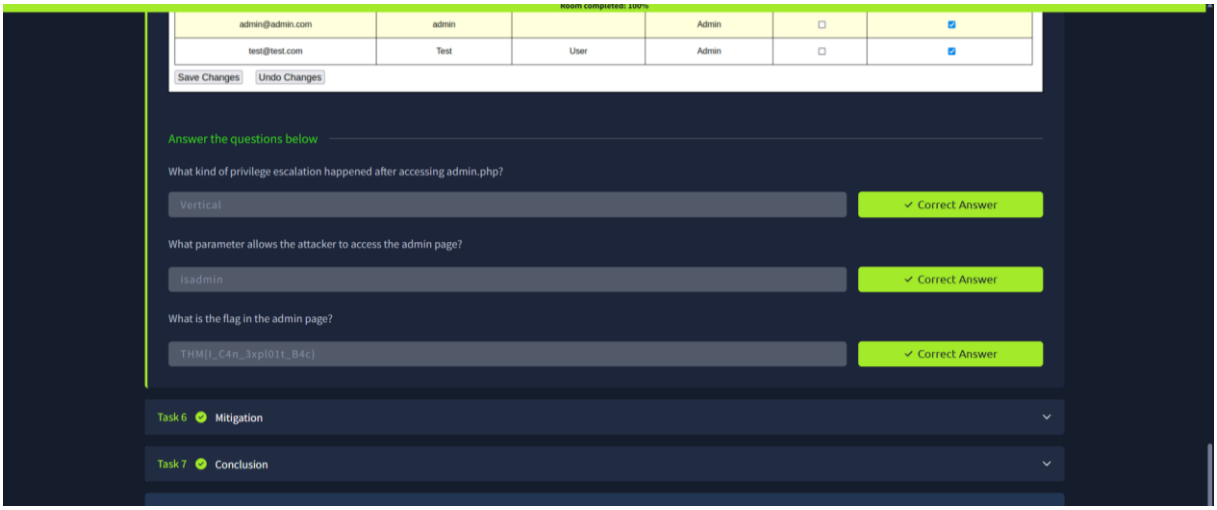


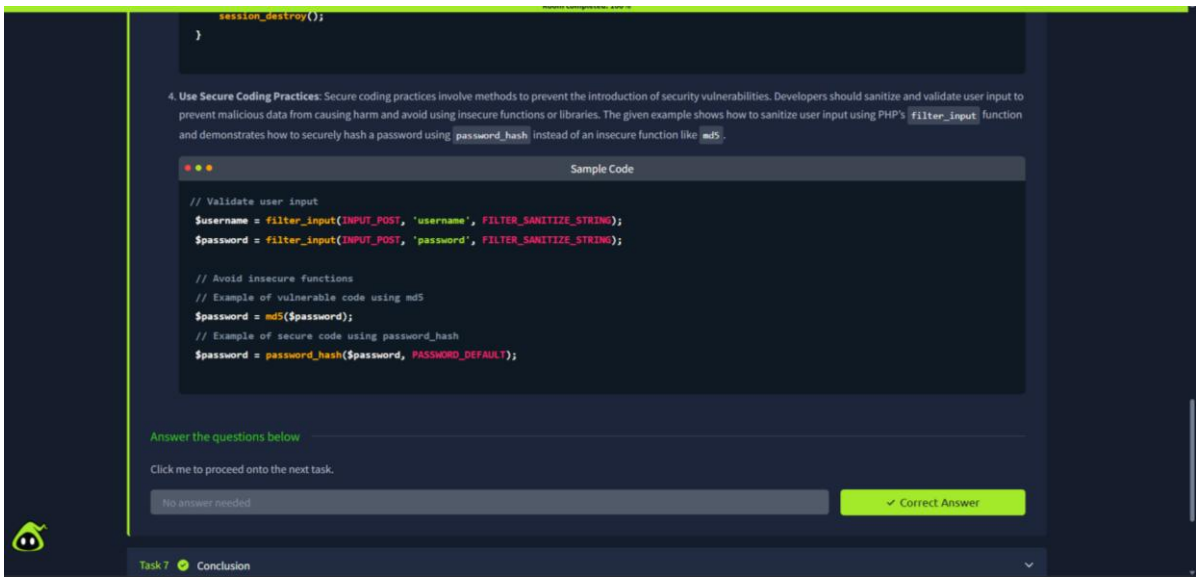


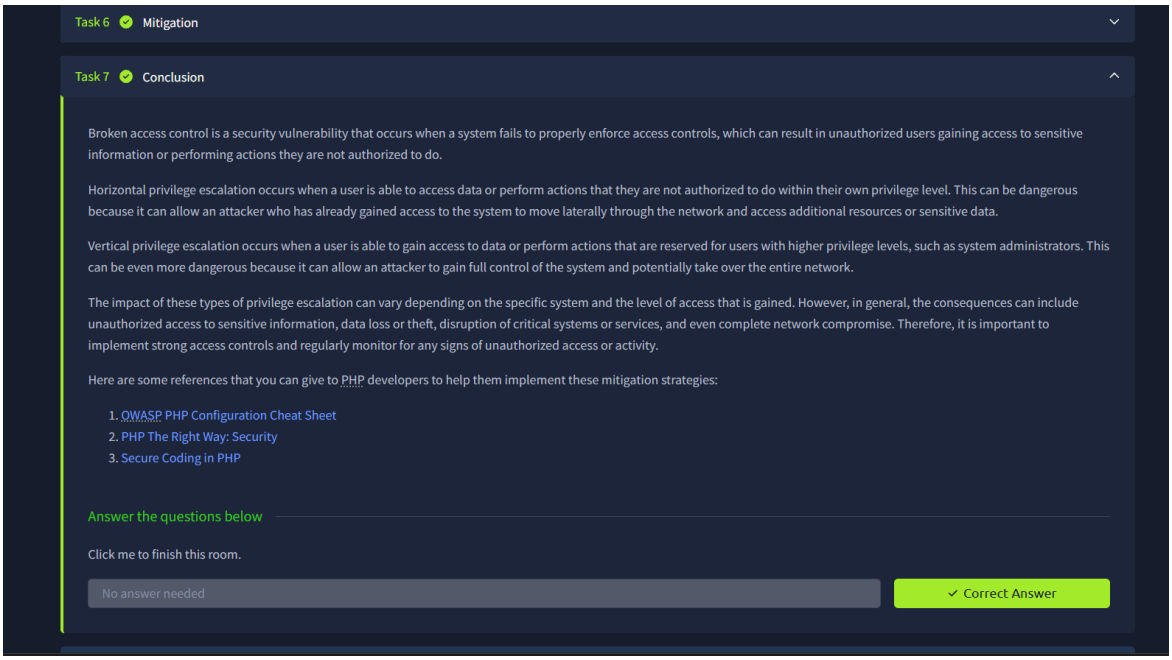

Room completed: 100%
These exploits can be prevented by implementing strong access control mechanisms and regularly reviewing and testing them to ensure they are functioning as intended.
Answer the questions below
What is IDOR?
Insecure direct object reference
✓ Correct Answer
What occurs when an attacker can access resources or data belonging to other users with the same level of access?
Horizontal privilege escalation
✓ Correct Answer
What occurs when an attacker can access resources or data from users with higher access levels?
Vertical privilege escalation
✓ Correct Answer
What is ABAC?
Attribute-Based Access Control
✓ Correct Answer
What is RBAC?
Role-Based Access Control
✓ Correct Answer
Task 3 Deploy the Machine


browser.
Welcome To VulnerableApp
Login to view your dashboard.
Login
Email
Enter email
Password
Enter your password
Submit
Don't have an account? Register
Keep in mind that the machine may take up to 5 minutes to spawn.
Answer the questions below
I have deployed the machine attached to the task.
No answer needed
✓ Correct Answer
Task 4 Assessing the Web Application
Task 5 Exploiting the Web Application
Task 6 Mitigation


Room completed: 100%
Answer the questions below
What is the type of server that is hosting the web application? This can be found in the response of the request in Burp Suite.
Apache
✓ Correct Answer
What is the name of the parameter in the JSON response from the login request that contains a redirect link?
redirect_link
✓ Correct Answer
What Burp Suite module allows us to capture requests and responses between ourselves and our target?
Proxy
✓ Correct Answer
What is the admin's email that can be found in the online users' table?
admin@admin.com
✓ Correct Answer
Task 5 Exploiting the Web Application
Task 6 Mitigation
Task 7 Conclusion
How likely are you to recommend this room to others?


admin@admin.com
admin
Admin
test@test.com
Test
User
Admin
Save Changes
Undo Changes
Answer the questions below
What kind of privilege escalation happened after accessing admin.php?
Vertical
✓ Correct Answer
What parameter allows the attacker to access the admin page?
isadmin
✓ Correct Answer
What is the flag in the admin page?
THM{I_C4n_3xpl0it_B4c}
✓ Correct Answer
Task 6 Mitigation
Task 7 Conclusion


session_destroy();
}
4. Use Secure Coding Practices: Secure coding practices involve methods to prevent the introduction of security vulnerabilities. Developers should sanitize and validate user input to prevent malicious data from causing harm and avoid using insecure functions or libraries. The given example shows how to sanitize user input using PHP's filter_input function and demonstrates how to securely hash a password using password_hash instead of an insecure function like md5.
Sample Code
// Validate user input
$username = filter_input(INPUT_POST, 'username', FILTER_SANITIZE_STRING);
$password = filter_input(INPUT_POST, 'password', FILTER_SANITIZE_STRING);
// Avoid insecure functions
// Example of vulnerable code using md5
$password = md5($password);
// Example of secure code using password_hash
$password = password_hash($password, PASSWORD_DEFAULT);
Answer the questions below
Click me to proceed onto the next task.
No answer needed
✓ Correct Answer
Task 7 Conclusion



6. Impact

If exploited in a real application, Broken Access Control may lead to the following consequences:

- **Unauthorized data disclosure:** Private account information, documents, or records may be exposed.
- **Account-level compromise:** An attacker may access or modify resources belonging to other users.
- **Privilege escalation:** A lower-privileged user may execute administrative functions.
- **Unauthorized modification:** User records, account settings, or business data may be changed.
- **Data deletion:** Missing authorization checks may allow unauthorized deletion of resources.
- **Business and compliance risks:** Exposure of sensitive data may cause financial loss, reputational damage, and regulatory consequences.

The actual severity depends on the resources affected, the privileges obtained, and the extent of unauthorized access demonstrated during testing.

7. Remediation

The following controls help prevent Broken Access Control vulnerabilities.

1. Enforce server-side authorization

Validate the user's permissions on every request that accesses protected data or performs a restricted action.

2. Verify resource ownership

Ensure that the authenticated user is authorized to access the requested object. Do not trust an object ID supplied by the client alone.

3. Apply role-based access control

Define clear permissions for roles such as standard user, moderator, and administrator. Verify these permissions on the server.

4. Follow the principle of least privilege

Grant users and services only the permissions required for their responsibilities.

5. Use deny-by-default authorization

Deny access unless an explicit authorization rule permits the requested operation.

6. Protect API endpoints

Apply authorization checks to API requests as well as normal webpage requests. Hiding a link or button is not sufficient.

7. Validate every sensitive operation

Verify authorization before reading, creating, updating, or deleting a protected resource.

8. Avoid trusting client-side controls

Do not use frontend logic, hidden fields, cookies controlled by the client, or predictable identifiers as the sole basis for authorization.

9. Log and monitor access violations

Record relevant unauthorized-access attempts and monitor suspicious patterns without unnecessarily logging sensitive information.

10. Perform regular security testing

Include horizontal and vertical privilege escalation tests in security reviews and regression testing.

8. What You Learned

Through this lab, I aimed to develop an understanding of Broken Access Control and its importance in web application security.

The key learning outcomes are:

- Understanding how authorization differs from authentication.
- Recognizing horizontal and vertical privilege escalation.
- Understanding the relationship between IDOR and Broken Access Control.
- Identifying user-controlled resource identifiers and sensitive endpoints.
- Understanding why authorization must be enforced on the server.
- Learning to document security testing with requests, responses, and screenshots.
- Understanding the impact of unauthorized access to sensitive resources.
- Learning practical remediation techniques, including resource ownership checks, least privilege, and deny-by-default policies.

Update these learning outcomes if any of the listed techniques were not covered during the actual lab.

Conclusion

Broken Access Control is a significant web application security risk because it can allow users to access resources or perform actions beyond their assigned permissions.

This report provides a structured approach to examining authorization enforcement, resource ownership, restricted endpoints, and privilege boundaries in an authorized lab environment. Effective remediation requires consistent server-side authorization checks, least-privilege permissions, deny-by-default policies, and regular security testing.

The final assessment should be based on the actual observations, screenshots, and completion status recorded during the TryHackMe exercise.

Report Prepared By: Rohit Kushwah
Organization: IqraSec Academy
Internship: Web Application Penetration Testing (WAPT)